

Modelagem da estrutura



Modelagem Estrutural de Sistemas

Este material apresenta os principais diagramas estruturais usados na modelagem de sistemas de software, cada um com um propósito específico para a organização e o entendimento da estrutura e das relações do sistema. Vamos detalhar os conceitos centrais de cada diagrama, abordando seus elementos, utilidades e exemplos práticos.

Diagrama de Objetos

O que é o Diagrama de Objetos?

O diagrama de objetos é uma representação visual dos objetos, ou seja, das instâncias das classes, em um momento específico do sistema. Ele oferece uma “foto” do sistema em ação, onde cada objeto tem valores concretos e específicos para seus atributos. Esse diagrama permite ver a relação entre os objetos de forma prática, indicando como as instâncias das classes estão interagindo em um cenário real.

Diferença entre Objetos e Classes

No contexto da modelagem, uma classe é um molde ou estrutura que define os atributos e comportamentos comuns a todos os objetos daquele tipo. Um objeto é uma instância dessa classe, uma representação específica com dados próprios.

Por exemplo:

Em uma biblioteca, a classe “Livro” define que todo livro tem um título e um ano de publicação.

Um objeto dessa classe poderia ser “livro1”, com valores como “Título: 1984” e “Ano: 1949”.

Utilização e Benefícios

O diagrama de objetos é fundamental para validar o design do sistema. Ele ajuda a visualizar como o sistema se comporta com dados reais, permitindo verificar se as classes criadas realmente atendem às necessidades. Esse diagrama também facilita a comunicação entre a equipe de desenvolvimento e stakeholders, tornando as ideias mais claras ao representar situações do mundo real.

Diagrama de Componentes

Conceito

O diagrama de componentes organiza e ilustra as peças fundamentais do sistema, chamadas de componentes. Cada componente representa uma parte importante do software, podendo ser módulos independentes, serviços, bibliotecas, entre outros. Além de representar os componentes em si, o diagrama mostra as interfaces (pontos de comunicação) entre eles e as dependências (relações de necessidade entre um componente e outro).

Estrutura e Elementos Principais

Componentes: São as peças principais do sistema, como um “serviço de login” ou uma “biblioteca de processamento de dados”.

Interfaces: Pontos pelos quais os componentes se conectam, oferecendo ou consumindo serviços de outros componentes. Uma interface pode representar, por exemplo, uma API que oferece acesso a determinados dados.

Dependências: Relações que indicam que um componente precisa de outro para funcionar. Elas são representadas com setas no diagrama, mostrando o fluxo de necessidade entre os componentes.

Exemplo de Aplicação

Imagine uma aplicação de previsão do tempo, onde:

O componente “Nuvem” oferece o serviço de “Precipitação”.

O componente “Ar” fornece o serviço de “Condensação”.

O componente “Lago” disponibiliza o serviço de “Evaporação”.

Neste exemplo, o diagrama de componentes mostra a relação de dependência entre esses serviços. Isso permite que os desenvolvedores vejam facilmente como as partes do sistema se interconectam e em que ordem os serviços devem ser implementados para garantir o funcionamento adequado do sistema.

Diagrama de Instalação

O que é o Diagrama de Instalação?

Também chamado de deployment diagram, este diagrama representa a distribuição física dos componentes de software e hardware de um sistema, permitindo entender como o sistema é implantado e organizado na infraestrutura real. Ele mostra quais componentes estão instalados em cada dispositivo ou servidor e como eles se conectam.

Elementos Principais

Nós (Nodes): São as unidades de hardware (como computadores, servidores ou dispositivos de rede) que suportam os componentes de software.

Componentes de Software: São os módulos ou programas que rodam nesses nós, como aplicativos ou serviços.

Exemplo de Estrutura

Suponha que estamos representando uma estrutura de sistema de uma empresa:

O nó “Computador do Usuário” executa um navegador de internet.

O nó “Servidor de Aplicação” hospeda uma aplicação web.

O nó “Servidor de Banco de Dados” armazena as informações da aplicação.

Esse diagrama facilita o entendimento de como cada parte do sistema é distribuída fisicamente e onde cada peça de software está instalada. Isso é

essencial para o gerenciamento de infraestrutura, uma vez que permite visualizar a relação entre o hardware e o software, prever desempenho e distribuir a carga de trabalho de maneira eficiente.

Diagrama de Pacotes

Função do Diagrama de Pacotes

O diagrama de pacotes organiza os elementos do sistema (como classes e interfaces) em pacotes, que são agrupamentos lógicos. Ele é extremamente útil para estruturar sistemas complexos, pois divide o sistema em partes mais gerenciáveis e com funções específicas, facilitando a compreensão e o gerenciamento.

Estrutura e Principais Elementos

Pacotes: São conjuntos de elementos agrupados com funções específicas, como classes, interfaces e até outros pacotes. Cada pacote representa uma área funcional do sistema.

Relações de Dependência: Indicadas por setas, as dependências mostram quais pacotes precisam de outros para operar.

Exemplo de Organização

Em um sistema de e-commerce, o diagrama de pacotes pode dividir o sistema em partes funcionais:

O pacote “Interface” agrupa todas as classes responsáveis pela interação com o usuário.

O pacote “Serviços” contém as classes que implementam as regras de negócio.

O pacote “Modelos” armazena as classes de dados que representam os produtos e clientes.

O pacote “Banco de Dados” centraliza a conexão com o sistema de armazenamento de dados.

Essas divisões facilitam a manutenção do sistema, pois organizam as funções e relações em módulos claros e bem definidos, permitindo localizar rapidamente as dependências e classes em cada pacote.

Diagrama de Estrutura

O que é o Diagrama de Estrutura?

Também chamado de diagrama de estrutura composta, este diagrama revela a estrutura interna de um classificador. Um classificador pode ser uma classe, um componente ou até mesmo um subsistema, e ele é mostrado no diagrama com suas partes internas e conexões entre elas.

Elementos Principais

Classificador: É o elemento principal do diagrama, que pode ser uma classe ou componente que precisa ser detalhado.

Partes Internas: São as subdivisões ou elementos internos do classificador, mostrando como ele é composto.

Portas: Pontos de interação entre o classificador e o ambiente externo.

Conectores: Relações entre as partes internas, que mostram como elas se conectam para formar o todo.

Exemplo de Estrutura Interna

Para uma classe “Gerente Loja”, podemos ter clientes divididos entre “Cliente Comum” e “Cliente VIP”, e cada um com suas instâncias de pedidos e itens. O diagrama de estrutura mostraria como essas partes interagem entre si, e como o Gerente Loja lida com as diferentes categorias de clientes e seus respectivos pedidos.

Esse diagrama é valioso para projetar e documentar a estrutura interna dos elementos de um sistema, facilitando a criação de componentes internos e mostrando como cada parte contribui para o funcionamento total.

Conclusão

Esses cinco tipos de diagramas oferecem visões diferentes e complementares sobre o sistema, desde o detalhamento dos objetos em um momento específico até a organização dos componentes físicos e lógicos. Cada diagrama facilita a análise e organização de partes específicas do sistema, ajudando a equipe a visualizar e planejar a estrutura completa de maneira eficiente.

Essas ferramentas são indispensáveis para um desenvolvimento bem organizado, claro e eficiente, especialmente em projetos de maior complexidade, onde a documentação e a organização são cruciais para o sucesso e a manutenção do sistema.

Conteúdo Bônus

Título: Diagrama De Componentes Na Prática | UML

Plataforma: YouTube

Canal: Pisani da Arch

Descrição: Neste vídeo, você explorará a fundo os diagramas de componentes, compreendendo sua importância na arquitetura de software. O conteúdo abrange para que serve um diagrama de componentes, a utilidade de seus conectores e como aplicá-los corretamente. A apresentação demonstra como esses diagramas em UML ajudam a visualizar as conexões e interações essenciais entre os elementos do sistema, proporcionando uma visão estratégica para aprimorar suas habilidades em arquitetura de software.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. Fundamentos da programação de computadores. 3.ed. Pearson: 2012.

BRAGA, P. H. Teste de software. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). Arquitetura de software. Pearson: 2017.

GALLOTTI, G. M. A. Qualidade de software. Pearson: 2015.

MEDEIROS, E. Desenvolvendo software com UML 2.0 definitivo. Pearson: 2004.

MORAIS, I. S. de. (Org.). Engenharia de software. Pearson: 2017.

PFLEEGER, S. L. Engenharia de software: teoria e prática. 2.ed. Pearson: 2003.

SOMMERVILLE, I. Engenharia de software. 10.ed. Pearson: 2019.